Universidad Nacional de Colombia

Programación Orientada a Objetos 2026-2

Actividad 3

Interfaz gráfica de usuario y gestión de eventos (Swing)

Ejercicios 8.2 (Notas) y 8.3 (Figuras geométricas) – Bermón Angarita

Estudiante: Santiago Herrera Martínez
Docente: Walter Hugo Arboleda Mazo

Fecha de entrega: jueves 22 de octubre de 2026

Repositorio GitHub: https://github.com/santiHM-cloud/actividad-3-POO

Contenido y enlaces de GitHub

Ejercicio	Contenido de este documento	Enlace GitHub
8.2 Notas (2.5 pts)	Interfaz, casos de uso, diagrama de clases, diagrama de objetos, código	https://github.com/santiHM-cloud/actividad-3-POO/tree/main/src/notas
8.3 Figuras (2.5 pts)	Interfaz, casos de uso, diagrama de clases, diagrama de objetos, código	https://github.com/santiHM-cloud/actividad-3-POO/tree/main/src/figuras

Ambos ejercicios incluyen su parte propuesta. Se programó en Java con Swing (JFrame, JPanel, JLabel, JTextField, JButton, JOptionPane) y el manejo de eventos se hace con ActionListener, como pide el texto guía.

Cómo ejecutar el proyecto

- Abrir la carpeta del repositorio en VS Code (con el Extension Pack for Java) y ejecutar Principal.java del paquete notas o figuras.
- Desde la terminal, dentro de la carpeta del repositorio: javac -encoding UTF-8 -d out src/notas/*.java src/figuras/*.java
- Luego: java -cp out notas.Principal (ejercicio 8.2) o java -cp out figuras.Principal (ejercicio 8.3)

Ejercicio 8.2 – Notas (interfaz gráfica de usuario)

Enunciado (resumen)

Programa con interfaz gráfica que permite ingresar cinco notas. Al pulsar el botón Calcular se muestran el promedio, la desviación estándar, la nota mayor y la nota menor; el botón Limpiar borra las notas ingresadas.

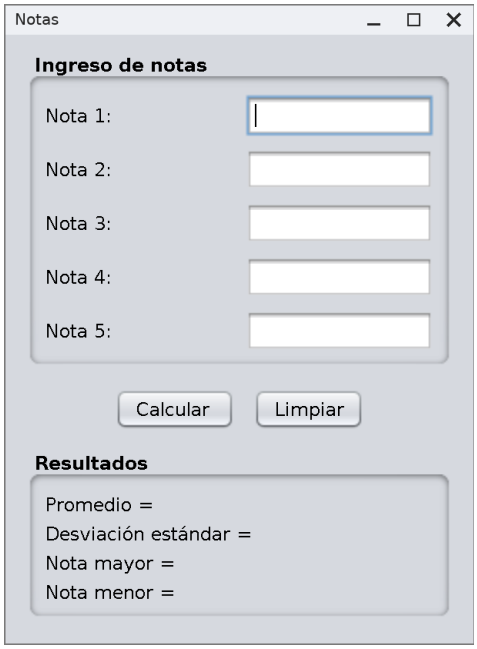
Parte propuesta (resuelta)

- Mostrar un mensaje de alerta cuando se ingresen datos que no sean numéricos.
- Es obligatorio ingresar las cinco notas: se muestra una alerta cuando falta alguna.

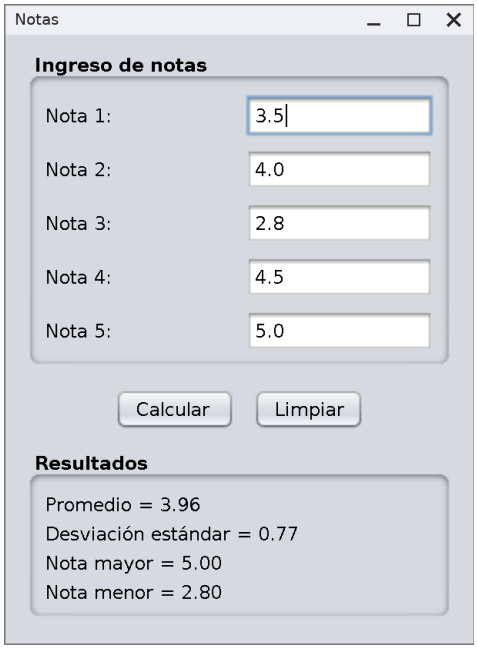
Decisiones de diseño

- La clase Notas (modelo) está separada de VentanaNotas (vista y eventos), y Principal solo arranca el programa.
- La desviación estándar es la poblacional (se divide entre n), igual que en el libro.
- Se aceptan decimales con punto o con coma (4,5 y 4.5), y se rechazan NaN e infinito.

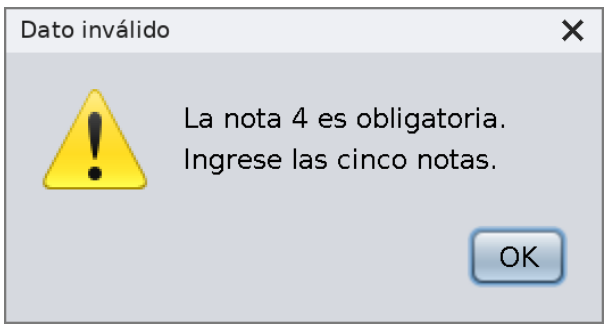
Imagen de la interfaz de usuario



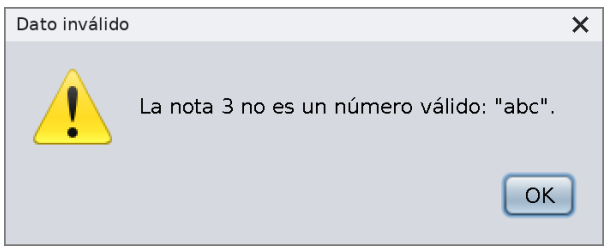
Ventana al iniciar



Resultado con las notas 3.5, 4.0, 2.8, 4.5 y 5.0

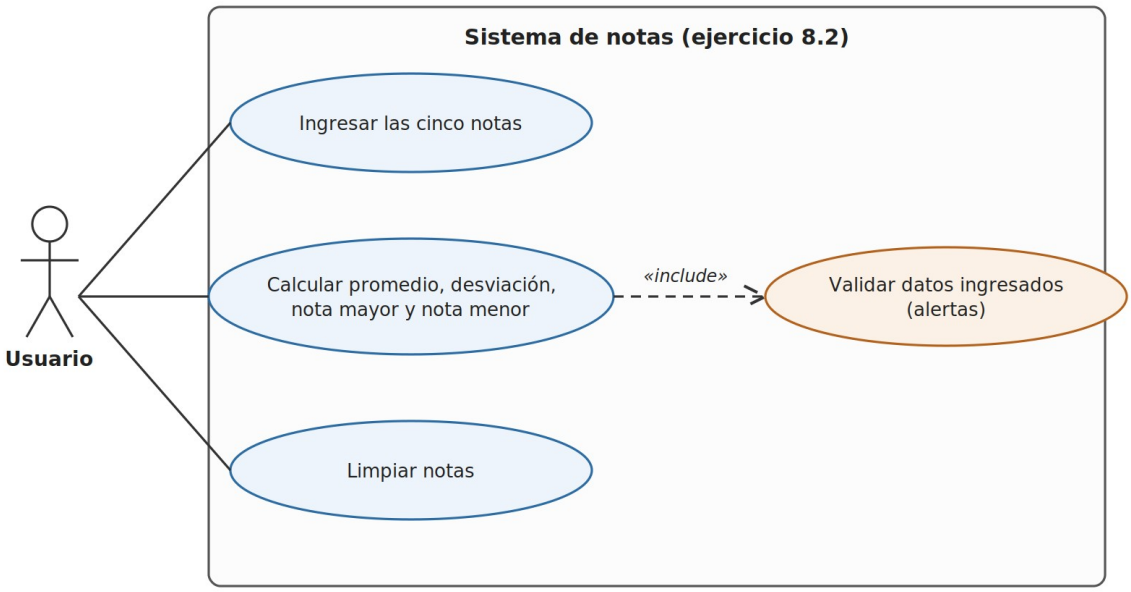


Alerta: falta una nota (Nota 4 vacía)



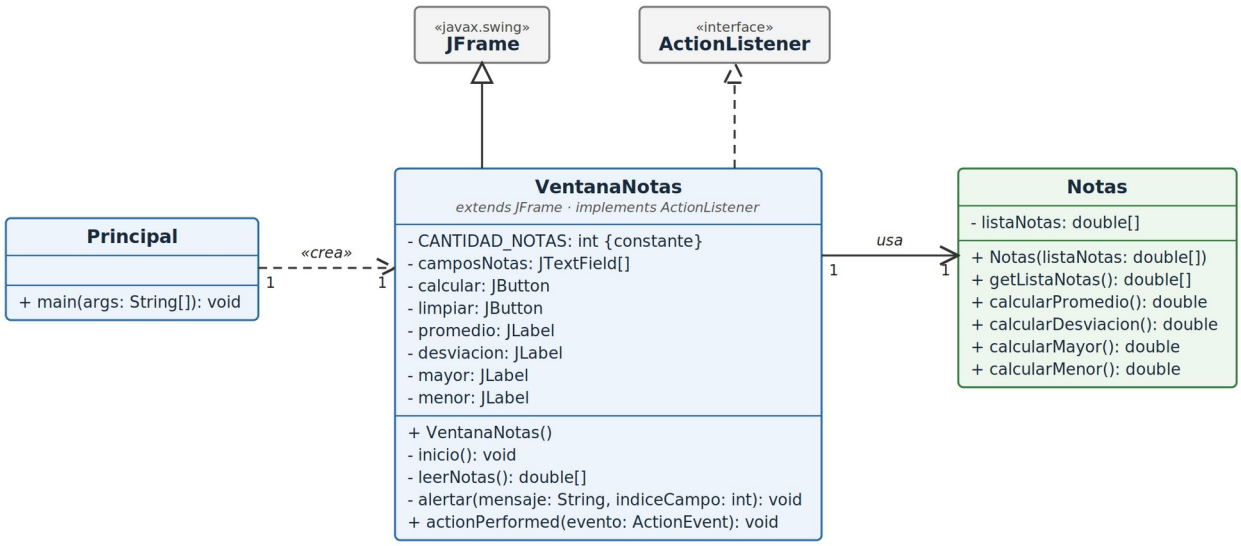
Alerta: dato no numérico ("abc")

Casos de uso



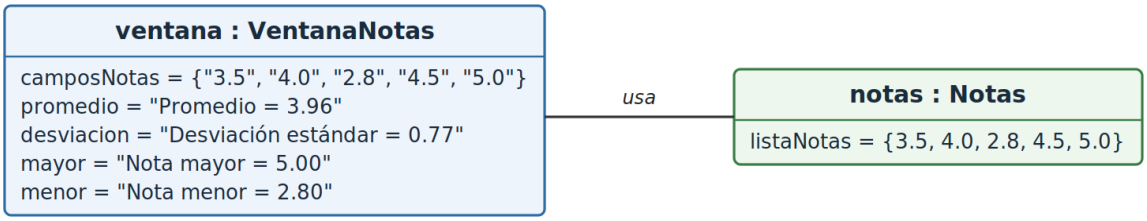
Caso de uso	Actor	Descripción
Ingresar las cinco notas	Usuario	El usuario escribe las cinco notas en los campos de texto.
Calcular promedio, desviación, mayor y menor	Usuario	Al pulsar Calcular se validan los datos, se crea el objeto Notas y se muestran los cuatro resultados con dos decimales.
Validar datos ingresados (include)	Sistema	Si una nota está vacía o no es numérica se muestra una alerta y no se calcula.
Limpiar notas	Usuario	Borra los campos y los resultados y deja el cursor en la primera nota.

Diagrama de clases



Principal crea una VentanaNotas (asociación 1 a 1). La ventana extiende JFrame, implementa ActionListener y usa un objeto Notas por cada cálculo.

Diagrama de objetos



Objetos en memoria después de calcular con las notas 3.5, 4.0, 2.8, 4.5 y 5.0: promedio 3.96, desviación estándar 0.77, mayor 5.00 y menor 2.80.

Ejercicio 8.3 – Figuras geométricas (gestión de eventos)

Enunciado (resumen)

Programa con interfaz gráfica para calcular el volumen y la superficie de varias figuras: cilindro (radio y altura), esfera (radio) y pirámide (base, altura y apotema), todo en centímetros. Se debe crear una jerarquía de clases para las figuras y gestionar los eventos de los botones con oyentes (ActionListener).

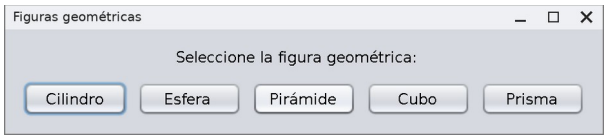
Parte propuesta (resuelta)

- Agregar el cálculo de otras figuras: cubo y prisma (se implementó un prisma rectangular de largo, ancho y altura).
- Agregar a cada figura su imagen: cada ventana muestra un dibujo de la figura con sus medidas (r, h, a, largo, ancho, alto), hecho con Java2D.

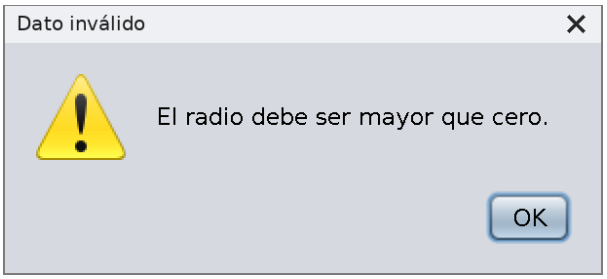
Decisiones de diseño

- FiguraGeometrica es una clase abstracta con volumen y superficie; Cilindro, Esfera, Piramide, Cubo y Prisma la extienden (herencia y polimorfismo).
- VentanaFigura es una ventana base abstracta: cada ventana concreta solo indica sus medidas y cómo crea su figura, así no se repite código.
- El volumen de la esfera usa $4/3 \cdot \pi \cdot r^3$ exacto. La pirámide es de base cuadrada: $V = base^2 \cdot h/3$ y $S = base^2 + 2 \cdot base \cdot apotema$.
- Se validan campos vacíos, textos no numéricos y medidas menores o iguales a cero con mensajes de alerta.

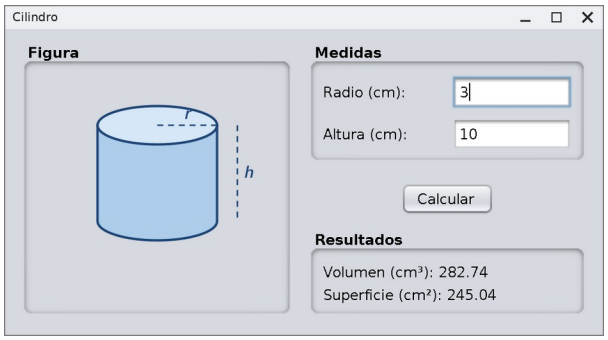
Imagen de la interfaz de usuario



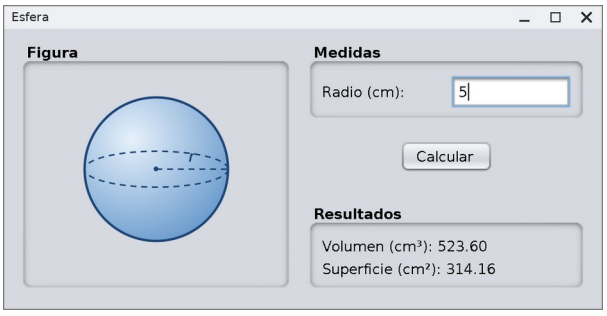
Ventana principal



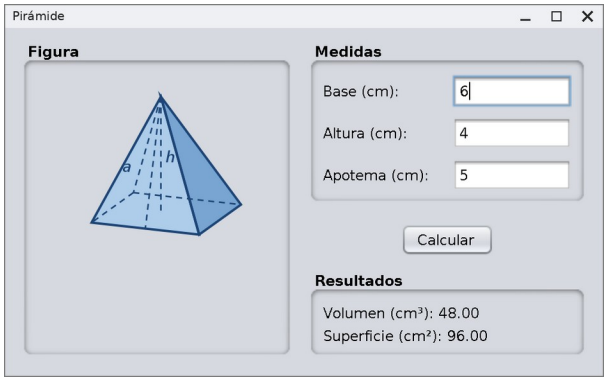
Alerta: radio negativo (-2)



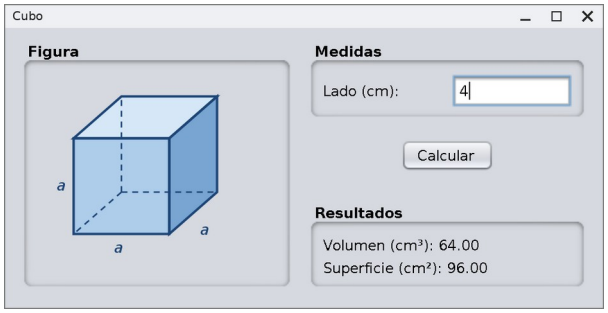
Cilindro (r = 3, h = 10)



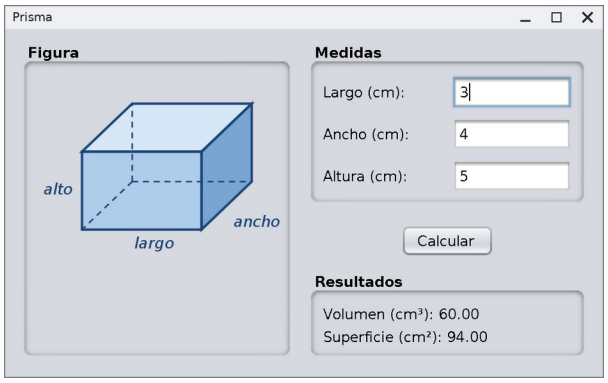
Esfera (r = 5)



Pirámide (base 6, altura 4, apotema 5)

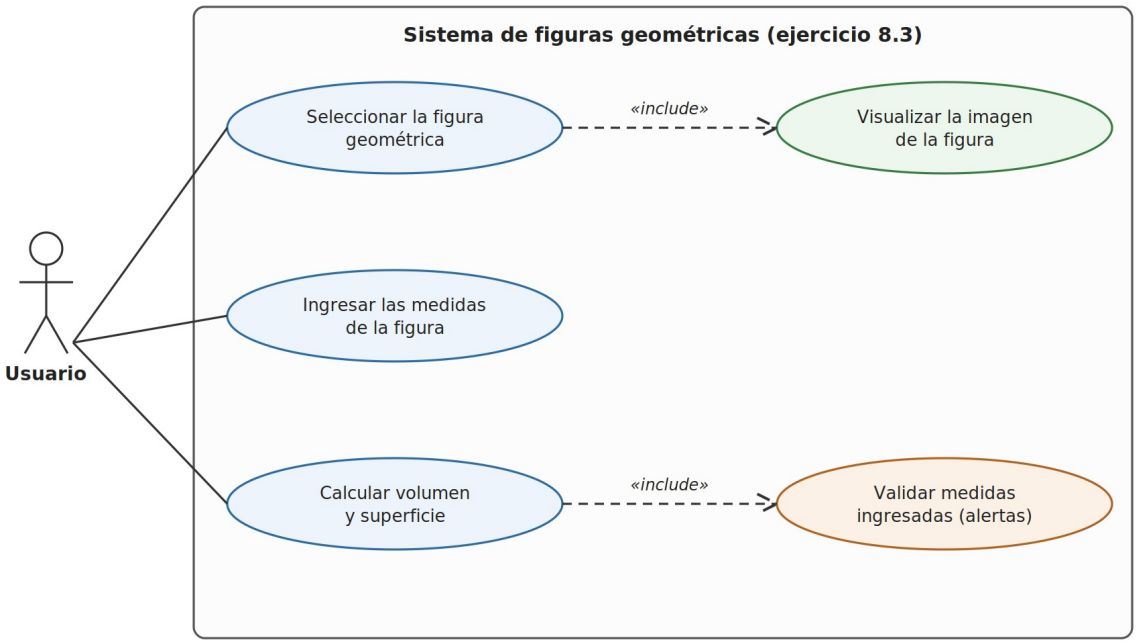


Cubo (lado 4)



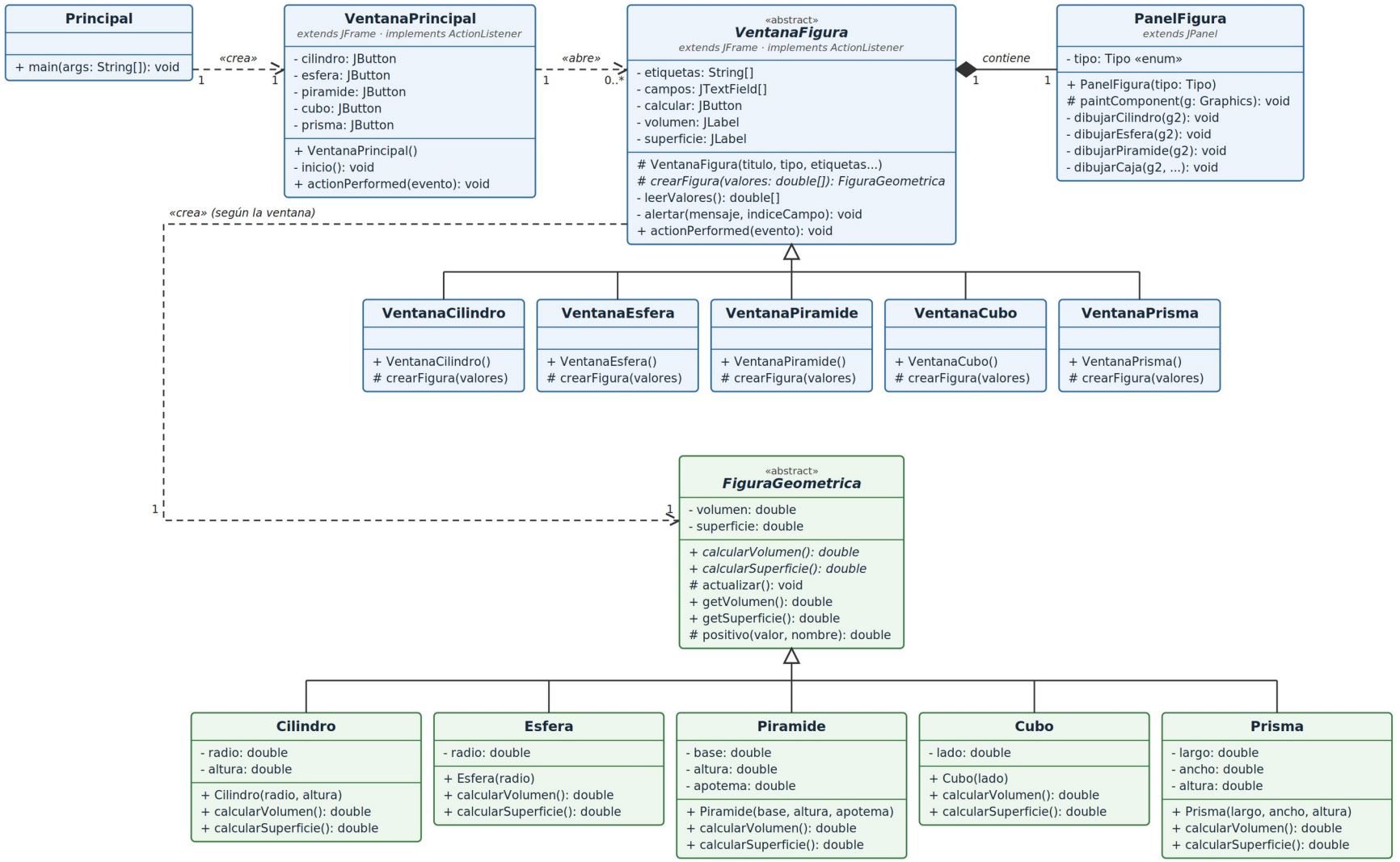
Prisma (3 × 4 × 5)

Casos de uso



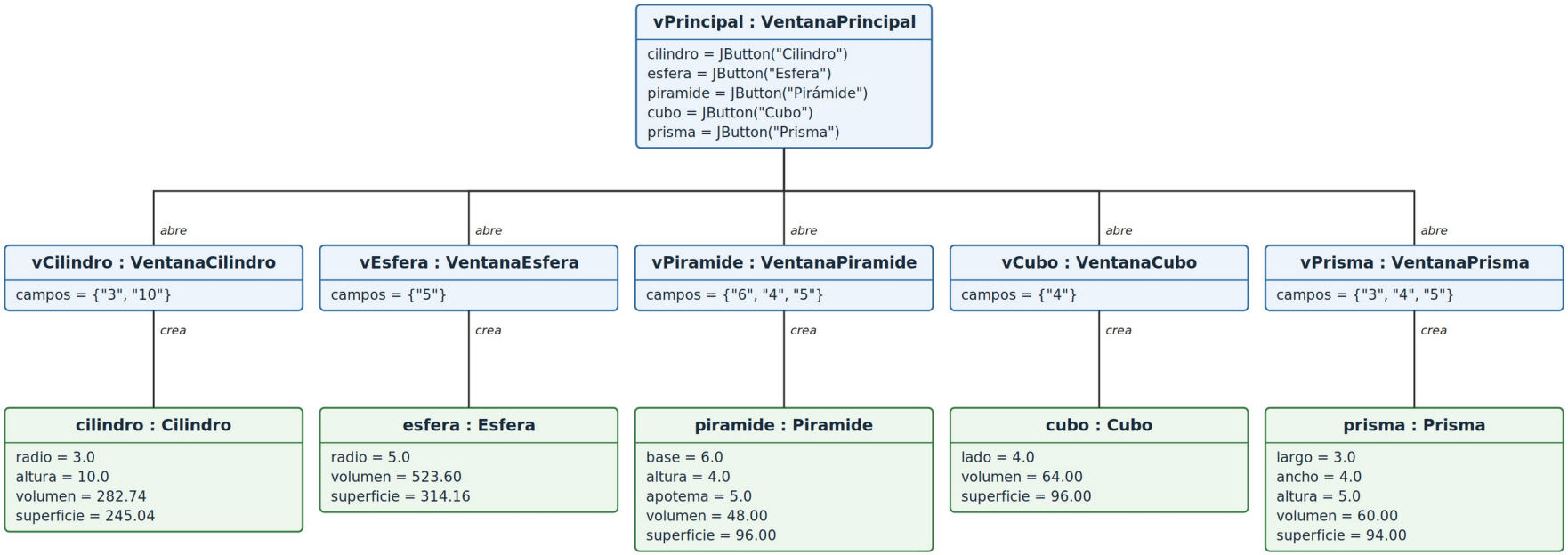
Caso de uso	Actor	Descripción
Seleccionar la figura geométrica	Usuario	Desde la ventana principal el usuario pulsa el botón de la figura; se abre su ventana.
Visualizar la imagen de la figura (include)	Sistema	Al abrirse, la ventana dibuja la figura con sus medidas.
Ingresar las medidas de la figura	Usuario	El usuario escribe las medidas en cm en los campos de la ventana.
Calcular volumen y superficie	Usuario	Al pulsar Calcular se crea la figura y se muestran volumen (cm^3) y superficie (cm^2).
Validar medidas ingresadas (include)	Sistema	Si hay campos vacíos, textos o medidas no positivas, muestra una alerta y no calcula.

Ejercicio 8.3 – Diagrama de clases



Principal crea VentanaPrincipal, que abre (0..*) ventanas de figuras. VentanaFigura es abstracta y contiene un PanelFigura con la imagen. Cada ventana concreta crea su figura, subclase de FiguraGeometrica.

Ejercicio 8.3 – Diagrama de objetos



Objetos tras calcular un caso de ejemplo en cada ventana: cilindro (3, 10), esfera (5), pirámide (6, 4, 5), cubo (4) y prisma (3, 4, 5).

Anexo – Código fuente

El mismo código está en el repositorio: https://github.com/santiHM-cloud/actividad-3-POO

Ejercicio 8.2 (paquete notas)

Archivo: src/notas/Notas.java

```
package notas;

/**
 * Modelo del ejercicio 8.2: guarda una lista de notas y calcula
 * el promedio, la desviación estándar, la nota mayor y la nota menor.
 */
public class Notas {

    private double[] listaNotas;

    public Notas(double[] listaNotas) {
        this.listaNotas = listaNotas.clone();
    }

    public double[] getListaNotas() {
        return listaNotas.clone();
    }

    public double calcularPromedio() {
        double suma = 0;
        for (double nota : listaNotas) {
            suma += nota;
        }
        return suma / listaNotas.length;
    }

    /** Desviación estándar poblacional (se divide entre n, como en el libro). */
    public double calcularDesviacion() {
        double promedio = calcularPromedio();
        double suma = 0;
        for (double nota : listaNotas) {
            suma += Math.pow(nota - promedio, 2);
        }
        return Math.sqrt(suma / listaNotas.length);
    }

    public double calcularMayor() {
        double mayor = listaNotas[0];
        for (double nota : listaNotas) {
            if (nota > mayor) {
                mayor = nota;
            }
        }
        return mayor;
    }

    public double calcularMenor() {
        double menor = listaNotas[0];
        for (double nota : listaNotas) {
            if (nota < menor) {
                menor = nota;
            }
        }
        return menor;
    }
}
```

Archivo: src/notas/VentanaNotas.java

```
package notas;

import java.awt.BorderLayout;
import java.awt.FlowLayout;
import java.awt.GridLayout;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import javax.swing.BorderFactory;
import javax.swing.JButton;
import javax.swing.JFrame;
```

```
import javax.swing.JLabel;
import javax.swing.JOptionPane;
import javax.swing.JPanel;
import javax.swing.JTextField;

/**
 * Ventana del ejercicio 8.2: pide cinco notas y muestra, en la parte
 * inferior, el promedio, la desviación estándar, la mayor y la menor.
 * Incluye la parte propuesta: alertas por datos vacíos o no numéricos.
 */
public class VentanaNotas extends JFrame implements ActionListener {

    private static final int CANTIDAD_NOTAS = 5;

    private JTextField[] camposNotas;
    private JButton calcular;
    private JButton limpiar;
    private JLabel promedio;
    private JLabel desviacion;
    private JLabel mayor;
    private JLabel menor;

    public VentanaNotas() {
        inicio();
        setTitle("Notas");
        pack();
        setLocationRelativeTo(null);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setResizable(false);
    }

    private void inicio() {
        setLayout(new BorderLayout(10, 10));
        ((JPanel) getContentPane()).setBorder(BorderFactory.createEmptyBorder(12, 12, 12, 12));

        // Zona superior: etiquetas y campos de las cinco notas
        JPanel panelNotas = new JPanel(new GridLayout(CANTIDAD_NOTAS, 2, 8, 8));
        panelNotas.setBorder(BorderFactory.createTitledBorder("Ingreso de notas"));
        camposNotas = new JTextField[CANTIDAD_NOTAS];
        for (int i = 0; i < CANTIDAD_NOTAS; i++) {
            panelNotas.add(new JLabel("Nota " + (i + 1) + ":"));
            camposNotas[i] = new JTextField(10);
            panelNotas.add(camposNotas[i]);
        }

        // Zona central: botones
        calcular = new JButton("Calcular");
        calcular.addActionListener(this);
        limpiar = new JButton("Limpiar");
        limpiar.addActionListener(this);
        JPanel panelBotones = new JPanel(new FlowLayout(FlowLayout.CENTER, 12, 4));
        panelBotones.add(calcular);
        panelBotones.add(limpiar);

        JPanel superior = new JPanel(new BorderLayout(0, 6));
        superior.add(panelNotas, BorderLayout.CENTER);
        superior.add(panelBotones, BorderLayout.SOUTH);

        // Zona inferior: resultados
        promedio = new JLabel("Promedio = ");
        desviacion = new JLabel("Desviación estándar = ");
        mayor = new JLabel("Nota mayor = ");
        menor = new JLabel("Nota menor = ");
        JPanel panelResultados = new JPanel(new GridLayout(4, 1, 0, 4));
        panelResultados.setBorder(BorderFactory.createTitledBorder("Resultados"));
        panelResultados.add(promedio);
        panelResultados.add(desviacion);
        panelResultados.add(mayor);
        panelResultados.add(menor);

        add(superior, BorderLayout.CENTER);
        add(panelResultados, BorderLayout.SOUTH);
    }

    /**
     * Lee las cinco notas de los campos. Si alguna está vacía o no es un
     * número, muestra una alerta y devuelve null.
     */
    private double[] leerNotas() {
```

```
        double[] valores = new double[CANTIDAD_NOTAS];
        for (int i = 0; i < CANTIDAD_NOTAS; i++) {
            String texto = camposNotas[i].getText().trim();
            if (texto.isEmpty()) {
                alertar("La nota " + (i + 1) + " es obligatoria.\nIngrese las cinco notas.", i);
                return null;
            }
            try {
                double valor = Double.parseDouble(texto.replace(',', '.'));
                if (Double.isNaN(valor) || Double.isInfinite(valor)) {
                    throw new NumberFormatException();
                }
                valores[i] = valor;
            } catch (NumberFormatException e) {
                alertar("La nota " + (i + 1) + " no es un número válido: \"" + texto + "\".", i);
                return null;
            }
        }
        return valores;
    }

    private void alertar(String mensaje, int indiceCampo) {
        JOptionPane.showMessageDialog(this, mensaje, "Dato inválido", JOptionPane.WARNING_MESSAGE);
        camposNotas[indiceCampo].requestFocus();
        camposNotas[indiceCampo].selectAll();
    }

    @Override
    public void actionPerformed(ActionEvent evento) {
        if (evento.getSource() == calcular) {
            double[] valores = leerNotas();
            if (valores != null) {
                Notas notas = new Notas(valores);
                promedio.setText("Promedio = " + String.format("%.2f", notas.calcularPromedio()));
                desviacion.setText("Desviación estándar = " + String.format("%.2f",
notas.calcularDesviacion()));
                mayor.setText("Nota mayor = " + String.format("%.2f", notas.calcularMayor()));
                menor.setText("Nota menor = " + String.format("%.2f", notas.calcularMenor()));
            }
        }
        if (evento.getSource() == limpiar) {
            for (JTextField campo : camposNotas) {
                campo.setText("");
            }
            promedio.setText("Promedio = ");
            desviacion.setText("Desviación estándar = ");
            mayor.setText("Nota mayor = ");
            menor.setText("Nota menor = ");
            camposNotas[0].requestFocus();
        }
    }
}
```

Archivo: src/notas/Principal.java

```
package notas;

import javax.swing.SwingUtilities;
import javax.swing.UIManager;

/** Punto de entrada del ejercicio 8.2. */
public class Principal {

    public static void main(String[] args) {
        try {
            for (UIManager.LookAndFeelInfo info : UIManager.getInstalledLookAndFeels()) {
                if ("Nimbus".equals(info.getName())) {
                    UIManager.setLookAndFeel(info.getClassName());
                    break;
                }
            }
        } catch (Exception e) {
            // Si Nimbus no está disponible se usa el aspecto por defecto
        }
        SwingUtilities.invokeLater(() -> new VentanaNotas().setVisible(true));
    }
}
```

Ejercicio 8.3 (paquete figuras)

Archivo: src/figuras/FiguraGeometrica.java

```
package figuras;

/**
 * Clase padre de la jerarquía del ejercicio 8.3. Guarda el volumen y la
 * superficie; cada figura define cómo se calculan.
 */
public abstract class FiguraGeometrica {

    private double volumen;
    private double superficie;

    public abstract double calcularVolumen();

    public abstract double calcularSuperficie();

    /** Las subclases lo llaman al final de su constructor. */
    protected void actualizar() {
        this.volumen = calcularVolumen();
        this.superficie = calcularSuperficie();
    }

    public double getVolumen() {
        return volumen;
    }

    public double getSuperficie() {
        return superficie;
    }

    /** Verifica que una medida sea mayor que cero. */
    protected static double positivo(double valor, String nombre) {
        if (valor <= 0) {
            throw new IllegalArgumentException(nombre + " debe ser mayor que cero.");
        }
        return valor;
    }
}
```

Archivo: src/figuras/Cilindro.java

```
package figuras;

public class Cilindro extends FiguraGeometrica {

    private double radio;
    private double altura;

    public Cilindro(double radio, double altura) {
        this.radio = positivo(radio, "El radio");
        this.altura = positivo(altura, "La altura");
        actualizar();
    }

    @Override
    public double calcularVolumen() {
        return Math.PI * Math.pow(radio, 2) * altura;
    }

    @Override
    public double calcularSuperficie() {
        return 2 * Math.PI * radio * altura + 2 * Math.PI * Math.pow(radio, 2);
    }
}
```

Archivo: src/figuras/Esfera.java

```
package figuras;

public class Esfera extends FiguraGeometrica {

    private double radio;

    public Esfera(double radio) {
        this.radio = positivo(radio, "El radio");
        actualizar();
    }
```

```
    @Override
    public double calcularVolumen() {
        return (4.0 / 3.0) * Math.PI * Math.pow(radio, 3);
    }

    @Override
    public double calcularSuperficie() {
        return 4 * Math.PI * Math.pow(radio, 2);
    }
}
```

Archivo: src/figuras/Piramide.java

```
package figuras;

/** Pirámide de base cuadrada. */
public class Piramide extends FiguraGeometrica {

    private double base;
    private double altura;
    private double apotema;

    public Piramide(double base, double altura, double apotema) {
        this.base = positivo(base, "La base");
        this.altura = positivo(altura, "La altura");
        this.apotema = positivo(apotema, "El apotema");
        actualizar();
    }

    @Override
    public double calcularVolumen() {
        return Math.pow(base, 2) * altura / 3;
    }

    @Override
    public double calcularSuperficie() {
        // área de la base + 4 triángulos laterales (base * apotema / 2 cada uno)
        return Math.pow(base, 2) + 2 * base * apotema;
    }
}
```

Archivo: src/figuras/Cubo.java

```
package figuras;

/** Figura propuesta: cubo. */
public class Cubo extends FiguraGeometrica {

    private double lado;

    public Cubo(double lado) {
        this.lado = positivo(lado, "El lado");
        actualizar();
    }

    @Override
    public double calcularVolumen() {
        return Math.pow(lado, 3);
    }

    @Override
    public double calcularSuperficie() {
        return 6 * Math.pow(lado, 2);
    }
}
```

Archivo: src/figuras/Prisma.java

```
package figuras;

/** Figura propuesta: prisma rectangular. */
public class Prisma extends FiguraGeometrica {

    private double largo;
    private double ancho;
    private double altura;

    public Prisma(double largo, double ancho, double altura) {
        this.largo = positivo(largo, "El largo");
```

```
        this.ancho = positivo(ancho, "El ancho");
        this.altura = positivo(altura, "La altura");
        actualizar();
    }

    @Override
    public double calcularVolumen() {
        return largo * ancho * altura;
    }

    @Override
    public double calcularSuperficie() {
        return 2 * (largo * ancho + largo * altura + ancho * altura);
    }
}
```

Archivo: src/figuras/PanelFigura.java

```
package figuras;

import java.awt.BasicStroke;
import java.awt.Color;
import java.awt.Dimension;
import java.awt.Font;
import java.awt.Graphics;
import java.awt.Graphics2D;
import java.awt.RadialGradientPaint;
import java.awt.RenderingHints;
import java.awt.geom.Area;
import java.awt.geom.Ellipse2D;
import java.awt.geom.Line2D;
import java.awt.geom.Point2D;
import java.awt.geom.Rectangle2D;
import javax.swing.BorderFactory;
import javax.swing.JPanel;

/**
 * Parte propuesta del ejercicio 8.3: dibuja la imagen de la figura
 * geométrica correspondiente a cada ventana.
 */
public class PanelFigura extends JPanel {

    public enum Tipo { CILINDRO, ESFERA, PIRAMIDE, CUBO, PRISMA }

    private static final Color RELLENO = new Color(173, 205, 235);
    private static final Color RELLENO_OSCURO = new Color(120, 165, 210);
    private static final Color RELLENO_CLARO = new Color(214, 231, 247);
    private static final Color TRAZO = new Color(30, 70, 120);
    private static final float[] PUNTEADO = {5f, 4f};

    private final Tipo tipo;

    public PanelFigura(Tipo tipo) {
        this.tipo = tipo;
        setPreferredSize(new Dimension(230, 210));
        setBorder(BorderFactory.createTitledBorder("Figura"));
    }

    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        Graphics2D g2 = (Graphics2D) g.create();
        g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING, RenderingHints.VALUE_ANTIALIAS_ON);
        g2.setRenderingHint(RenderingHints.KEY_TEXT_ANTIALIASING, RenderingHints.VALUE_TEXT_ANTIALIAS_ON);
        g2.setFont(new Font(Font.SANS_SERIF, Font.ITALIC, 13));
        switch (tipo) {
            case CILINDRO: dibujarCilindro(g2); break;
            case ESFERA: dibujarEsfera(g2); break;
            case PIRAMIDE: dibujarPiramide(g2); break;
            case CUBO: dibujarCaja(g2, 45, 80, 80, 80, 40, -35, "a", "a", "a"); break;
            case PRISMA: dibujarCaja(g2, 52, 90, 100, 65, 42, -40, "largo", "alto", "ancho"); break;
        }
        g2.dispose();
    }

    private void trazoSolido(Graphics2D g2) {
        g2.setColor(TRAZO);
        g2.setStroke(new BasicStroke(2f));
    }
```

```
    private void trazoPunteado(Graphics2D g2) {
        g2.setColor(TRAZO);
        g2.setStroke(new BasicStroke(1.3f, BasicStroke.CAP_BUTT, BasicStroke.JOIN_MITER, 10f, PUNTEADO,
0f));
    }

    private void dibujarCilindro(Graphics2D g2) {
        Area cuerpo = new Area(new Rectangle2D.Double(65, 68, 100, 80));
        cuerpo.add(new Area(new Ellipse2D.Double(65, 132, 100, 32)));
        g2.setColor(RELLENO);
        g2.fill(cuerpo);
        trazoSolido(g2);
        g2.draw(cuerpo);
        Ellipse2D tapa = new Ellipse2D.Double(65, 52, 100, 32);
        g2.setColor(RELLENO_CLARO);
        g2.fill(tapa);
        trazoSolido(g2);
        g2.draw(tapa);
        trazoPunteado(g2);
        g2.draw(new Line2D.Double(115, 68, 165, 68));
        g2.draw(new Line2D.Double(182, 68, 182, 148));
        g2.drawString("r", 138, 63);
        g2.drawString("h", 188, 112);
    }

    private void dibujarEsfera(Graphics2D g2) {
        Ellipse2D esfera = new Ellipse2D.Double(55, 45, 120, 120);
        g2.setPaint(new RadialGradientPaint(new Point2D.Double(95, 80), 95f,
                new float[] {0f, 1f}, new Color[] {new Color(232, 242, 252), new Color(104, 154, 205)}));
        g2.fill(esfera);
        trazoSolido(g2);
        g2.draw(esfera);
        trazoPunteado(g2);
        g2.draw(new Ellipse2D.Double(55, 90, 120, 30));
        g2.draw(new Line2D.Double(115, 105, 175, 105));
        g2.fill(new Ellipse2D.Double(113, 103, 4, 4));
        g2.drawString("r", 142, 99);
    }

    private void dibujarPiramide(Graphics2D g2) {
        int[] ax = {60, 150, 185, 95};   // base: frente-izq, frente-der, fondo-der, fondo-izq
        int[] ay = {150, 160, 135, 125};
        int cx = 118, cy = 45;           // vértice superior
        // caras visibles
        g2.setColor(RELLENO);
        g2.fillPolygon(new int[] {cx, ax[0], ax[1]}, new int[] {cy, ay[0], ay[1]}, 3);
        g2.setColor(RELLENO_OSCURO);
        g2.fillPolygon(new int[] {cx, ax[1], ax[2]}, new int[] {cy, ay[1], ay[2]}, 3);
        trazoSolido(g2);
        g2.drawPolygon(new int[] {cx, ax[0], ax[1]}, new int[] {cy, ay[0], ay[1]}, 3);
        g2.drawPolygon(new int[] {cx, ax[1], ax[2]}, new int[] {cy, ay[1], ay[2]}, 3);
        // aristas ocultas, altura y apotema
        trazoPunteado(g2);
        g2.drawLine(cx, cy, ax[3], ay[3]);
        g2.drawLine(ax[3], ay[3], ax[0], ay[0]);
        g2.drawLine(ax[3], ay[3], ax[2], ay[2]);
        g2.drawLine(cx, cy, cx, 140);
        g2.drawLine(cx, cy, 105, 155);
        g2.drawString("h", cx + 4, 100);
        g2.drawString("a", 86, 108);
    }

    /** Dibuja una caja (cubo o prisma rectangular) con perspectiva oblicua. */
    private void dibujarCaja(Graphics2D g2, int x, int y, int w, int h, int dx, int dy,
                             String etqAncho, String etqAlto, String etqProf) {
        g2.setColor(RELLENO);
        g2.fillRect(x, y, w, h);
        g2.setColor(RELLENO_CLARO);
        g2.fillPolygon(new int[] {x, x + dx, x + w + dx, x + w}, new int[] {y, y + dy, y + dy, y}, 4);
        g2.setColor(RELLENO_OSCURO);
        g2.fillPolygon(new int[] {x + w, x + w + dx, x + w + dx, x + w},
                new int[] {y, y + dy, y + h + dy, y + h}, 4);
        trazoSolido(g2);
        g2.drawRect(x, y, w, h);
        g2.drawPolygon(new int[] {x, x + dx, x + w + dx, x + w}, new int[] {y, y + dy, y + dy, y}, 4);
        g2.drawPolygon(new int[] {x + w, x + w + dx, x + w + dx, x + w},
                new int[] {y, y + dy, y + h + dy, y + h}, 4);
        trazoPunteado(g2);
```

```
        g2.drawLine(x + dx, y + h + dy, x, y + h);
        g2.drawLine(x + dx, y + h + dy, x + w + dx, y + h + dy);
        g2.drawLine(x + dx, y + h + dy, x + dx, y + dy);
        g2.drawString(etqAncho, x + w / 2 - 6, y + h + 16);
        g2.drawString(etqAlto, x - 8 - 6 * etqAlto.length(), y + h / 2 + 4);
        g2.drawString(etqProf, x + w + dx / 2 + 6, y + h + dy / 2 + 18);
    }
}
```

Archivo: src/figuras/VentanaFigura.java

```
package figuras;

import java.awt.BorderLayout;
import java.awt.FlowLayout;
import java.awt.GridLayout;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import javax.swing.BorderFactory;
import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JOptionPane;
import javax.swing.JPanel;
import javax.swing.JTextField;

/**
 * Ventana base de las figuras: muestra la imagen de la figura, los campos
 * de medidas, el botón Calcular y los resultados. Cada subclase solo indica
 * qué medidas pide y cómo crea su figura.
 */
public abstract class VentanaFigura extends JFrame implements ActionListener {

    private final String[] etiquetas;
    private final JTextField[] campos;
    private final JButton calcular;
    private final JLabel volumen;
    private final JLabel superficie;

    protected VentanaFigura(String titulo, PanelFigura.Tipo tipo, String... etiquetas) {
        this.etiquetas = etiquetas;
        setTitle(titulo);
        setLayout(new BorderLayout(10, 10));
        ((JPanel) getContentPane()).setBorder(BorderFactory.createEmptyBorder(12, 12, 12, 12));

        JPanel formulario = new JPanel(new GridLayout(etiquetas.length, 2, 8, 8));
        formulario.setBorder(BorderFactory.createTitledBorder("Medidas"));
        campos = new JTextField[etiquetas.length];
        for (int i = 0; i < etiquetas.length; i++) {
            formulario.add(new JLabel(etiquetas[i]));
            campos[i] = new JTextField(8);
            formulario.add(campos[i]);
        }

        calcular = new JButton("Calcular");
        calcular.addActionListener(this);
        JPanel panelBoton = new JPanel(new FlowLayout(FlowLayout.CENTER));
        panelBoton.add(calcular);

        volumen = new JLabel("Volumen (cm³): ");
        superficie = new JLabel("Superficie (cm²): ");
        JPanel resultados = new JPanel(new GridLayout(2, 1, 0, 4));
        resultados.setBorder(BorderFactory.createTitledBorder("Resultados"));
        resultados.add(volumen);
        resultados.add(superficie);

        JPanel derecha = new JPanel(new BorderLayout(0, 8));
        derecha.add(formulario, BorderLayout.NORTH);
        derecha.add(panelBoton, BorderLayout.CENTER);
        derecha.add(resultados, BorderLayout.SOUTH);

        add(new PanelFigura(tipo), BorderLayout.WEST);
        add(derecha, BorderLayout.CENTER);

        pack();
        setLocationByPlatform(true);
        setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);
        setResizable(false);
    }
```

```
    /** Cada ventana crea su figura con las medidas ingresadas. */
    protected abstract FiguraGeometrica crearFigura(double[] valores);

    private double[] leerValores() {
        double[] valores = new double[campos.length];
        for (int i = 0; i < campos.length; i++) {
            String nombre = etiquetas[i].replace(":", "");
            String texto = campos[i].getText().trim();
            if (texto.isEmpty()) {
                alertar("Falta ingresar: " + nombre + ".", i);
                return null;
            }
            try {
                double valor = Double.parseDouble(texto.replace(',', '.'));
                if (Double.isNaN(valor) || Double.isInfinite(valor)) {
                    throw new NumberFormatException();
                }
                valores[i] = valor;
            } catch (NumberFormatException e) {
                alertar(nombre + " no es un número válido: \"" + texto + "\".", i);
                return null;
            }
        }
        return valores;
    }

    private void alertar(String mensaje, int indiceCampo) {
        JOptionPane.showMessageDialog(this, mensaje, "Dato inválido", JOptionPane.WARNING_MESSAGE);
        campos[indiceCampo].requestFocus();
        campos[indiceCampo].selectAll();
    }

    @Override
    public void actionPerformed(ActionEvent evento) {
        if (evento.getSource() == calcular) {
            double[] valores = leerValores();
            if (valores == null) {
                return;
            }
            try {
                FiguraGeometrica figura = crearFigura(valores);
                volumen.setText("Volumen (cm³): " + String.format("%.2f", figura.getVolumen()));
                superficie.setText("Superficie (cm²): " + String.format("%.2f", figura.getSuperficie()));
            } catch (IllegalArgumentException e) {
                JOptionPane.showMessageDialog(this, e.getMessage(), "Dato inválido",
JOptionPane.WARNING_MESSAGE);
            }
        }
    }
}
```

Archivo: src/figuras/VentanaCilindro.java

```
package figuras;

public class VentanaCilindro extends VentanaFigura {

    public VentanaCilindro() {
        super("Cilindro", PanelFigura.Tipo.CILINDRO, "Radio (cm):", "Altura (cm):");
    }

    @Override
    protected FiguraGeometrica crearFigura(double[] valores) {
        return new Cilindro(valores[0], valores[1]);
    }
}
```

Archivo: src/figuras/VentanaEsfera.java

```
package figuras;

public class VentanaEsfera extends VentanaFigura {

    public VentanaEsfera() {
        super("Esfera", PanelFigura.Tipo.ESFERA, "Radio (cm):");
    }

    @Override
    protected FiguraGeometrica crearFigura(double[] valores) {
```

```
        return new Esfera(valores[0]);
    }
}
```

Archivo: src/figuras/VentanaPiramide.java

```
package figuras;

public class VentanaPiramide extends VentanaFigura {

    public VentanaPiramide() {
        super("Pirámide", PanelFigura.Tipo.PIRAMIDE, "Base (cm):", "Altura (cm):", "Apotema (cm):");
    }

    @Override
    protected FiguraGeometrica crearFigura(double[] valores) {
        return new Piramide(valores[0], valores[1], valores[2]);
    }
}
```

Archivo: src/figuras/VentanaCubo.java

```
package figuras;

public class VentanaCubo extends VentanaFigura {

    public VentanaCubo() {
        super("Cubo", PanelFigura.Tipo.CUBO, "Lado (cm):");
    }

    @Override
    protected FiguraGeometrica crearFigura(double[] valores) {
        return new Cubo(valores[0]);
    }
}
```

Archivo: src/figuras/VentanaPrisma.java

```
package figuras;

public class VentanaPrisma extends VentanaFigura {

    public VentanaPrisma() {
        super("Prisma", PanelFigura.Tipo.PRISMA, "Largo (cm):", "Ancho (cm):", "Altura (cm):");
    }

    @Override
    protected FiguraGeometrica crearFigura(double[] valores) {
        return new Prisma(valores[0], valores[1], valores[2]);
    }
}
```

Archivo: src/figuras/VentanaPrincipal.java

```
package figuras;

import java.awt.BorderLayout;
import java.awt.GridLayout;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import javax.swing.BorderFactory;
import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JPanel;
import javax.swing.SwingConstants;

/** Ventana principal del ejercicio 8.3: un botón por cada figura geométrica. */
public class VentanaPrincipal extends JFrame implements ActionListener {

    private JButton cilindro;
    private JButton esfera;
    private JButton piramide;
    private JButton cubo;
    private JButton prisma;

    public VentanaPrincipal() {
        inicio();
        setTitle("Figuras geométricas");
        pack();
        setLocationRelativeTo(null);
```

```
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setResizable(false);
    }

    private void inicio() {
        setLayout(new BorderLayout(0, 12));
        ((JPanel) getContentPane()).setBorder(BorderFactory.createEmptyBorder(14, 14, 14, 14));

        JLabel instruccion = new JLabel("Seleccione la figura geométrica:", SwingConstants.CENTER);

        cilindro = new JButton("Cilindro");
        esfera = new JButton("Esfera");
        piramide = new JButton("Pirámide");
        cubo = new JButton("Cubo");
        prisma = new JButton("Prisma");
        JPanel botones = new JPanel(new GridLayout(1, 5, 8, 0));
        for (JButton boton : new JButton[] {cilindro, esfera, piramide, cubo, prisma}) {
            boton.addActionListener(this);
            botones.add(boton);
        }

        add(instruccion, BorderLayout.NORTH);
        add(botones, BorderLayout.CENTER);
    }

    @Override
    public void actionPerformed(ActionEvent evento) {
        if (evento.getSource() == cilindro) {
            new VentanaCilindro().setVisible(true);
        }
        if (evento.getSource() == esfera) {
            new VentanaEsfera().setVisible(true);
        }
        if (evento.getSource() == piramide) {
            new VentanaPiramide().setVisible(true);
        }
        if (evento.getSource() == cubo) {
            new VentanaCubo().setVisible(true);
        }
        if (evento.getSource() == prisma) {
            new VentanaPrisma().setVisible(true);
        }
    }
}
```

Archivo: src/figuras/Principal.java

```
package figuras;

import javax.swing.SwingUtilities;
import javax.swing.UIManager;

/** Punto de entrada del ejercicio 8.3. */
public class Principal {

    public static void main(String[] args) {
        try {
            for (UIManager.LookAndFeelInfo info : UIManager.getInstalledLookAndFeels()) {
                if ("Nimbus".equals(info.getName())) {
                    UIManager.setLookAndFeel(info.getClassName());
                    break;
                }
            }
        } catch (Exception e) {
            // Si Nimbus no está disponible se usa el aspecto por defecto
        }
        SwingUtilities.invokeLater(() -> new VentanaPrincipal().setVisible(true));
    }
}
```